Coursework 2 — Data Analytics of Chess Games

1. Overview

The aim of this coursework is to develop a simple, data-intensive application in Python 3.

This coursework contributes to **50%** of the total mark for the course.

This is a **group project**, and you will have to identify each group member's contribution. The final submission must consist of a solution comprising a report, source code, and an executable. Each group should have a minimum of 2 and a maximum of 3 students.

The **learning objective** of this coursework is for students to develop proficiency in advanced programming concepts, stemming from both object-oriented and functional programming paradigms, and to apply these programming skills to a concrete application of moderate size. Design choices regarding tools and libraries chosen for the implementation need to be justified in the accompanying report.

This coursework will develop personal abilities in using modern scripting languages as a "glueware" to build, configure, and maintain a moderately complex application and deepen the understanding of integrating components.

2. Analysing Chess Games

In this coursework, we will be using data provided by lichess.org, a free open-source chess server. Lichess.org keeps a publically available[1] database of all rated chess games played on the platform. We will be using a small sample of that data in this coursework. See the data sample at `https://database.lichess.org/` to see what the data for each game looks like.

You do not need to know how to play chess in order to complete this coursework, as the analysis will not concern the games themselves, but the metadata regarding those games (e.g., player names, their ratings, when was the game played, ...).

The datasets for this task can be found at `https://marko.doko.foo/F2[01]SC/CW2/datasets`. Your application should be able to handle all the datasets found there. As a challenge, see if you are able to process a full month's dataset from `https://database.lichess.org`. All the datasets are Zstandard compressed plain text files containing multiple chess games in PGN format.

The data analysis application should be developed in Python 3, using appropriate libraries for input, data processing and visualisation. If you are not sure where to begin searching for appropriate libraries, see Section 3 for some ideas.

The application must provide the functionality described in the rest of this section.

2.1. Data preprocessing

The data provided to us by lichess.org contains a lot of information that is of no interest to us. On the other hand, some data that is of interest to us are not recorded explicitly. In order not to keep the unnecessary data, as well as to have all the data we are interested in readily available, we will *preprocess* the data to make it suitable for our purpose.

The following are the data we need to keep for each game:

- **Player names** together with the information on who had white/black pieces. These values are recoreded in game headers named `White` and `Black`.

[1] Under the Creative Commons CC0 license

- **Pre-game player ratings**, i.e., ratings of each player at the beginning of the game. These values are recorded in game heeaders named `WhiteElo` and `BlackElo`.
- **Post-game player ratings**, i.e. ratings of each player after the conclusion of the game. These values are not recorded directly in the provided data, but can be calculated by adding the values recodred in `WhiteRatingDiff` and `BlackRatingDiff` to the ratings recorder in the `WhiteElo` and `BlackElo` headers, respectively.
- **Game rating**, which we define as the average pre-game rating of the players playing the game. This value is not recorded directly in the provided data, but can be calculated by averaging the values recorded in the `WhiteElo` and `BlackElo` headers.
- The **opening** featured in the game (i.e., the name under which the initial moves of the game are known). The header `Opening` contains the opening's name, often including a particular opening variation. For example, in "Englund Gambit: Stockholm Variation" and "Caro-Kann Defense: Hillbilly Attack" the opening names are "Englund Gambit" and "Carro-Kann Defense", while "Stockholm Variation" and "Hillbilly Attack" are the respective opening variations. In general, if a variation is recorded, the name of the variation is separated from the opening's name by a colon (`:`). We are only interested in keeping the main opening names, not the variations.
- **Time control** of the game. The header `TimeControl` contains values of the form initial + increment, where initial is the time (in seconds) each player's clock starts with at the beginning of the game, and increment is the time (in seconds) added to the player's clock after each move. The games are categorised in various time controls depending on the *estimated game duration*, calculated as initial + 40 × increment. We will be interested in the games belonging to the *bullet*, *blitz*, and *rapid* time controls, classified as follows
  - bullet: $30 \leqslant$ estimated game duration < 180;
  - blitz: $180 \leqslant$ estimated game duration < 480;
  - rapid: $480 \leqslant$ estimated game duration < 1500.

  Keep only the games that fall under one of those three time controls, as well as the information which time control the game belongs to.
- **Timestamp** recording when the game was played. This information is spread across two headers, `UTCDate` and `UTCTime`. Combine them into a single timestamp to make sorting the games chronologically easier.

The program must be able to process the data provided by lichess.org and output a file containing the data we need (as described above) in a format conducive for executing the data analysis tasks described in the rest of this section. The exact format of the processed data file is left for you to choose; it will depend on how you decide to process the data.

The program must support the following command line invocation:

```
cw2.py --raw-data=in --outfile=out
```

where `in` is the location of the file containing the data provided by lichess.org, and `out` is the location of the file in which the preprocessed data will be saved.

2.2. Opening popularity

We want to analyse the popularity of openings across different time controls and ratings. The program will display a bar chart displaying openings that have been used in at least 1% of games within the user-specified time control and game rating range (see Section 2.1 for the definition of game rating). The names of the openings should be listed on the x-axis, and the correspodning percentages of games that featured that opening should be on the y-axis.

The program must support the following command line invocation:

```
cw2.py --datafile=filename --popular-openings --time-control=TC --min-rating=X --max-rating=Y
```

where `filename` is the location of the file containing the preprocessed data, `TC` is either `bullet`, `blitz`, or `rapid`, and `X` and `Y` are integers representing lower and upper bounds for the game rating, respectively. (Even though a game's rating might not be an integer, when doing this analysis we will only concern ourselves with specifying rating intervals bounded by integers.) The last three command line arguments are optional. If time control is not specified, games from all time controls should be analysed. If a lower or rating bound is left unspecified, the rating interval should be considered unbounded on the side of the unspecified bound.

2.3. Most improved players

In this task we want to identify the most improved players (i.e., the players who improved their rating the most) within the time period covered by the dataset. The overall change of the a player's rating can be calculated by subtracting their pre-game rating recorded in the first game they played from their post-game rating recorded in the last game they played.

The program will display the list of ten most improved players for the specified time control, ordered by how much each player improved (most improved one on top). Besides the player names, their initial and final ratings should be displayed, as well as the number of games they played.

The program must support the following command line invocation:

```
cw2.py --datafile=filename --top10 --time-control=name
```

where `filename` is the location of the file containing the preprocessed data, and `TC` is either `bullet`, `blitz`, or `rapid`.

2.4. Player similarity

When analysing data, it's often the case that we are interested in classifying objects according to some kind of similarity metric. In this task, we are going to take a look at two similarity metrics for players:

1. The "distance" between their preferences for playing different time formats. Let x_{bu}, x_{bl}, and x_{ra}, be the percentages of bullet, blitz, and rapid games played by player x. We measure the distance between players x and y using the formula

   $$d_{\text{tc}}(x, y) = \sqrt{(x_{\text{bu}} - y_{\text{bu}})^2 + (x_{\text{bl}} - y_{\text{bl}})^2 + (x_{\text{ra}} - y_{\text{ra}})^2}\,.$$

   The smaller the distance between the players, the more similar we consider them to be.

2. Their favourite opening percentage as black, i.e., the percentage of games played as black that featured their most-played opening as black. In this case, we only consider the players whose favourite openings as black (i.e., the opening they play most often when plaing with black pieces) is the same. The players are considered more similar the closer their favourite opening percentage with black pieces is to each other, i.e., the smaller the absolute difference between those two percentages is.

The program will display the list of ten most similar players to a given player.

The program must support the following command line invocations:

```
cw2.py --datafile=filename --players-similar-to=name --time-formats
cw2.py --datafile=filename --players-similar-to=name --black-openings
```

where `filename` is the location of the file containing the preprocessed data, and `name` is the name of the player whose similar players we are looking for. The `-time-formats` and `-black-openings` options select whether to find similar players based on their preferences for time formats or preferences for openings with black pieces, respectively.

To get full marks on this part of the coursework, **the solution should be modular with respect to the player similarity metrics**, i.e., it should be possible to add a support for a new similarity metric with minimal changes to the code; in particular it should not be necessary to change the subroutine that sorts the players according to the similarity to the chosen player. This could, for example, be achieved by implementing a sorting function that takes a similarity metric as an argument.

2.5. Graphical user interface (GUI)

Develop a simple GUI which will allow users to run all the tasks specified above. There should be clear GUI elements allowing users to select any of the above-mentioned preprocessing and analysis tasks, together with selecting necessary data files and processing other user inputs. The user should be able to complete multiple tasks during the single GUI session.

Additionally, when it comes to time controls, openings, and player names, the GUI should restrict the user to inputting only legal values, i.e., openings and player names should be restricted to the ones that actually appear in the data set (this could, for example, be presented as a, potentially searchable, drop-down list of valid entries), and time controls can only be bullet, blitz, or rapid.

The program will run GUI if run without any command line arguments, i.e., simply invoking `cw2.py` will start the GUI.

2.6. Error handling

The program must handle errors gracefully and clearly communicate to the user what went wrong. The list of errors that has to be taken into account includes, but is not limited to: unknown command line arguments, bad values (e.g., non-existing player names, openings, or time formats), non-existing/unreadable/inaccessible data files, data files containting incorrectly formatted data. Depending on your implementation, there might be additional possible run-time errors, and it is your responsibility to handle them appropriately.

When run in the command-line mode, the program should terminate after reporting the error, unless it is possible to recover without user intervention.

When run in the GUI mode, the program will report any errors (through the GUI!), but let the user continue operating the program.

3. Useful libraries

Here are some libraries that you might find useful for implementing this coursework. **You are not required to use any of them, nor are you limited to using only what is listed here.** Consider this a helpful shortlist and a starting point for your own exploration of possible solutions.

- `python-chess` – a chess library for Python. In particular, note the documentation section on "skimming".
- `compression.zstd` – the standard library module for processing Zstandard compressed files
- `argparse` – the standard library module for processing command line arguments
- `pandas` – data analysis library
- `matplotlib` – visualisation library (you can use it to plot charts)
- `tkinter` – a GUI library

4. Report Format

The report should be up to 12 pages long and use the following format (if you need space for additional screenshots, put them into an appendix, not counting against the page limit, but don't rely on the screenshots in your discussion):

1. **Introduction:** state the purpose of the report, your remit, and any assumptions you have made during the development process.
2. **Requirements checklist:** clearly state which requirements are implemented and which are not; this should be in the form of a table (as shown in Appendix A) listing all the required features, and stating whether the functionality is fully implemented, partially implemented, or missing.
3. **Design Considerations:** describe the basic design decisions you have made in developing your code, covering code organisation, choice of data structures, GUI design, performance-relevant choices etc. In particular, make sure to **design clear class and function interfaces** and document them in the report (see the next point).
4. **Developer guide:** describe your application design and main areas of code in order to help another developer understand your work and how they might continue the development. In particular, point out
   - how is your code organized in classes/modules and which class/module is responsible for which functionlaity;
   - specification of the preprocessed data format.
5. **Generative AI usage statement:** if you used generative AI while working on this coursework (see Section 7 for GenAI policy on this course), describe how was it included in your workflow. In particular, include any prompts and model's responses as appendices. (Appendices do not count towards the report's page limit.)
6. **Reflections on programming language and implementation:** based on your experience in implementing this application, reflect on which language features and technologies have been most helpful, identify limitations of your application and suggest ways how to overcome these limitations. Also, reflect on the usability of the language (or scripting languages in general) for this application domain, and on its wider applicability.
7. **What did we learn from CW1?** A short discussion on lessons learnt from the feedback given on CW1 and a discussion how you integrated this feedback into CW2. Cover both coding and report writing, and possibly more.
8. **Conclusion:** reflect on what you are most proud of in the application and what you would have liked to have done differently.
9. **Group member contributions:** a brief summary of each group member's contribution to the project
10. **References:** a list of references used in this report and in the implementation. (This section does not count toward the report's page limit.)

5. Video Demonstration Format

The video demonstration must showcase all the implemented and partially implemented features of the GUI part of the application, matching the requirement checklist provided in the report. There should be a voiceover audio explaining the demonstrated features. **Features not demonstrated in the video will be considered not implemented.**

Do not concern yourself with making a high-quality production video. All that is important is that the functionality of the application is clearly visible and explained.

The video showcasing the use of the application is **mandatory** (barring any mitigating circumstances). **Without the video the submission is incomplete** and may be marked as 0 points.

6. Submission and Marking Scheme

The coursework submission consists of

1. A zip archive containing
   - the source code, and
   - the report (in pdf format).
2. A video demonstration, as explained in Section 5. Either upload the video on Canvas or provide a OneDrive link to the video in the report.
3. If deemed necessary, you might be asked to give an in-person demonstration of your submission.

The standard penalty of 30% of the maximum available mark applies to late submissions. No submissions will be accepted after 5 working days beyond the submission deadline. Submission must be through Canvas. **This coursework is worth 50%** of the final mark. The marking scheme for this coursework can be seen below.

Marking Scheme	
Criterion	**Marks**
CLI functionality	
Data preprocessing	10
Opening popularity	10
Most improved players	10
Player similarity (any functional implementation 10 marks; full marks if modularity detailed in Section 2.4 is achieved	15
GUI functionality	
Provides functional interface for implemented features	5
Restricts user input to legal values where possible	5
Other coursework elements	
Error handling (dealing with errorneous and malformed input)	5
Code organisation and quality (portable coding, clear interfaces, sufficient comments, sensible variable/object names, good code/module/class/function design)	20
Report quality (Content and communication, design considerations, reflections, developer guide, feature checklist)	15
Video demonstration of running the GUI with explanation as voice-over audio	5
Total marks	**100**

7. Generative AI Usage Policy

Our policy on the use of generative AI tools in F21SC is as follows:

You may use GenAI to complete specified tasks you then evaluate this output yourself with no GenAI assistance to provide the assessment submission.

This policy corresponds to Level 6 of the University GenAI usage levels.

Please note that you are responsible for the quality of the code you submit, and you have to be able to provide a detailed developer's guide as well as **be able to discuss and explain the submitted code**, regardless of how much you relied on generative AI.

8. Academic Integrity

Your attention is drawn to the university policy on academic misconduct and to the Contract Cheating Risk and Avoidance Guide (links available via Canvas).

This is a **group project** and you will have to identify the contributions of each group member in the report. Each group member needs to contribute a substantial implementation task to the project.

The code you submit must be your own, or if some text or code in the coursework has been taken from sources outside the material provided for this course (e.g., textbooks, blog/reddit/stack overflow posts, LLM output), you must give a clear reference to the source in comments in the code or the report.

Failure to clearly reference work that has been obtained from other sources, or copying the words or code of another student, is plagiarism.

Asking a third party to complete coursework on your behalf, and then submitting it as your own work, is contract cheating.

You must never give a copy of your coursework writing or code to another student outside of your group, and you must always refuse any request from another student outside of your group for a copy of your coursework. Sharing your coursework with another student outside your group is collusion.

Suspected plagiarism, contract cheating or collusion will be referred to the School Discipline Committee for investigation. This will delay your feedback, and if you are found guilty of misconduct then it will result in a penalty such as voiding the course or being withdrawn from the university.

Please don't do any of the things above! If you have any concerns about potential academic misconduct, please ask your local course leader for guidance.

For details see University's academic misconduct policy and Student Guide to Academic Integrity.

A. Feature Checklist Example

Here is an example feature checklist, listing all the features that will be marked and their implementation status. Partially implemented features come with a small note describing the known issues.

FEATURE	STATUS
CLI functionality	
Data preprocessing	**fully implemented**
Opening popularity	**fully implemented**
Most improved players	**fully implemented**
Player similarity	**partially implemented** – Only the "favorite opening" similarity metric is implementd
GUI functionality	
Functional interface for implemented features	**partially implemented** – All analysis features supported. No support for running preprocessing from GUI.
User input restricted to legal values where possible	**not implemented**

The feature checklist in your report must list exactly the features listed in the above example. Of course, the status column should reflect the state of your submission.